TECHNICAL DATA ARCHITECTURE SPECIFICATION

Database Specification Document

PDF Vault & Digital Asset Storage System Data Models

Document ID	DBS-PDFV-2026-V1	**Version**	1.0.0 (Production Ready)
Release Date	October 20, 2026	**Prepared By**	Data Architecture & Backend Engineering
Primary ORM	SQLAlchemy 2.0 (Declarative Mapped Models)	**Development RDBMS**	SQLite 3 (WAL Mode Enabled)
Target Production RDBMS	PostgreSQL 16+ (AWS RDS / Cloud SQL)	**Schema Revision**	Rev 2.4 (Multi-Tenant Scoped)

1. Database Architecture & Design Rationale

The PDF Vault database architecture is engineered around the principle of strict multi-tenant isolation, referential integrity, and decoupled binary asset storage. Rather than storing large binary data (BLOBs) inside relational database tables—which causes severe database bloat, degrades buffer pool performance, and complicates backups—the system utilizes a high-performance hybrid architecture:

- **Relational Metadata Engine:** SQLAlchemy 2.0 models manage structured metadata (user accounts, folder taxonomies, file names, byte sizes, MIME content types, and foreign key relationships) in an optimized relational store.
- **Decoupled Binary Storage:** The physical file payloads are persisted in dedicated, structured filesystem directories (or cloud object stores like AWS S3 / Google Cloud Storage) under randomized, collision-resistant UUID4 filenames.
- **Multi-Tenant Boundary:** Every record in the system—whether a document, folder, image category, or image asset—is linked via foreign key to the `users.id` primary key, guaranteeing complete tenant segregation.

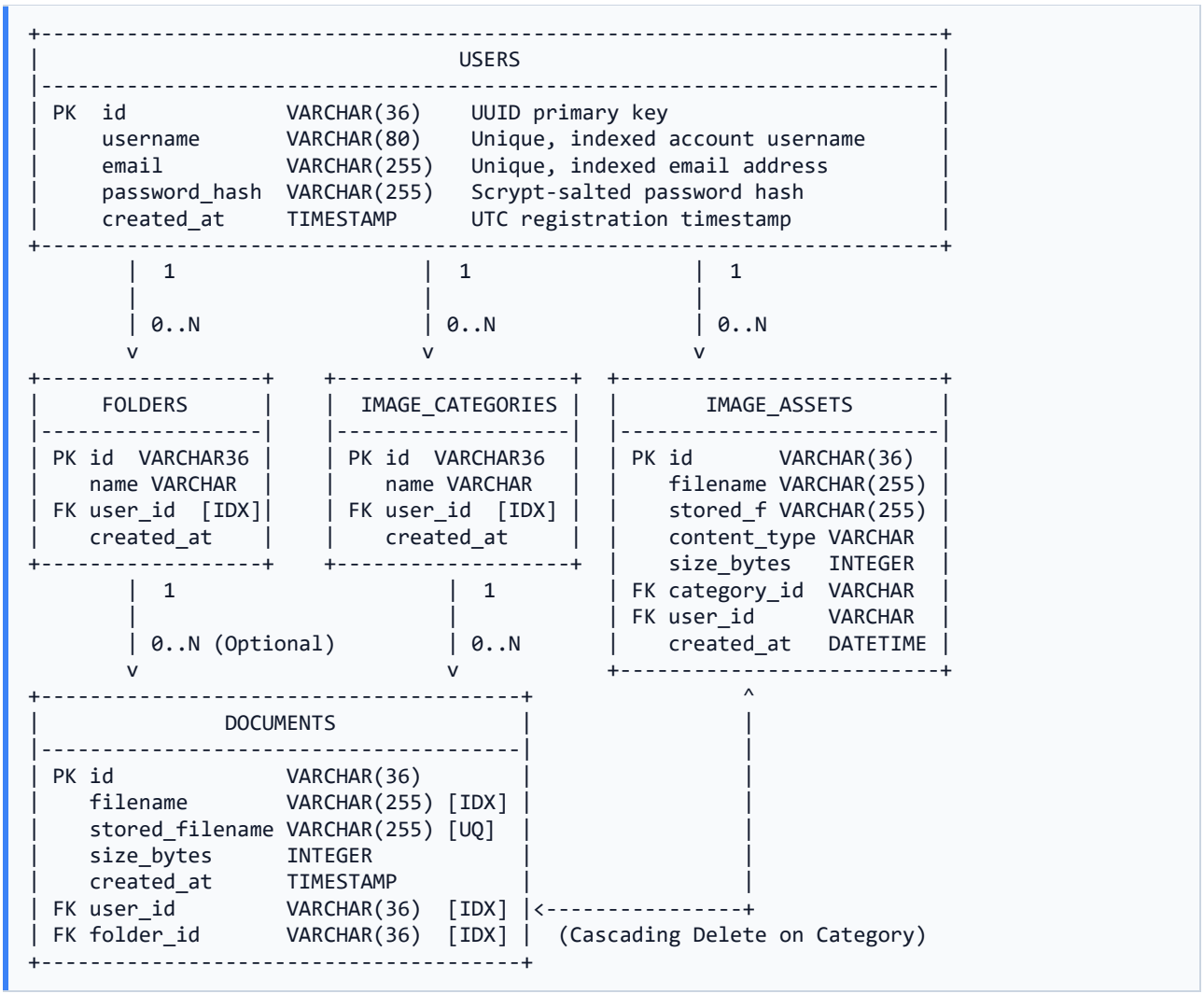
Development vs. Production Database Strategy

> The platform currently utilizes SQLite 3 for zero-configuration local development. The codebase is fully compatible with PostgreSQL 16+ without application-level changes, requiring only an update to the `DATABASE_URL` connection string in `.env`.

2. Entity-Relationship (ER) Model & Relationship Cardinality

The conceptual data model consists of five core entities interlinked through foreign key constraints:

2.1 Visual Entity-Relationship Architecture



2.2 Relationship Cardinality & Integrity Rules

Primary Table (1)	Related Table (N)	Foreign Key Field	Cardinality	On Delete Action	Business Behavior
users	folders	folders.user_id	1 : 0..N	CASCADE (DB) / RESTRICT	A user owns multiple document folders.
users	documents	documents.user_id	1 : 0..N	CASCADE (DB)	A user owns multiple PDF documents.
folders	documents	documents.folder_id	1 : 0..N	SET NULL	Safe unlinking: deleting folder sets folder_id to NULL. PDFs are preserved.
users	image_categories	image_categories.user_id	1 : 0..N	CASCADE (DB)	A user owns multiple image categories.
image_categories	image_assets	image_assets.category_id	1 : 0..N	CASCADE (App & DB)	Cascading: deleting category deletes all images & disk files.

Primary Table (1)	Related Table (N)	Foreign Key Field	Cardinality	On Delete Action	Business Behavior
users	image_assets	image_assets.user_id	1 : 0..N	CASCADE (DB)	A user owns multiple image assets across categories.

3. Comprehensive Data Dictionary

The technical specifications for each table in the schema are detailed below:

3.1 Table Name: `users`

Stores authenticated user credentials, tenant identity, and account timestamps.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
id	VARCHAR(36)	PRIMARY KEY	NOT NULL	UUID4 hex string	Unique immutable tenant identifier generated at registration.
username	VARCHAR(80)	UNIQUE, INDEX	NOT NULL	None	User display handle; alphanumeric 3-80 characters.
email	VARCHAR(255)	UNIQUE, INDEX	NOT NULL	None	User email address; stored in normalized lowercase.
password_hash	VARCHAR(255)	None	NOT NULL	None	Scrypt hash string: `scrypt$b64_salt$b64_digest`.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
created_at	TIMESTAMP WITH TIMEZONE	None	NOT NULL	UTC datetime.now	System account creation timestamp in UTC.

3.2 Table Name: `folders`

Defines user-specific directory categories for organizing PDF documents.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
id	VARCHAR(36)	PRIMARY KEY	NOT NULL	UUID4 hex string	Unique identifier for the document folder.
name	VARCHAR(120)	None	NOT NULL	None	Human-readable folder title (e.g. 'Tax 2026', 'Contracts').
user_id	VARCHAR(36)	FOREIGN KEY, INDEX	NOT NULL	None	References `users.id`; defines tenant ownership.
created_at	TIMESTAMP WITH TIMEZONE	None	NOT NULL	UTC datetime.now	Timestamp of folder creation in UTC.

3.3 Table Name: `documents`

Maintains metadata for stored PDF documents, linking to physical disk storage.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
id	VARCHAR(36)	PRIMARY KEY	NOT NULL	UUID4 hex string	Unique identifier for the document record.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
filename	VARCHAR(255)	INDEX	NOT NULL	None	Original user-facing file name (e.g. 'Q3_Financials.pdf').
stored_filename	VARCHAR(255)	UNIQUE	NOT NULL	None	Physical disk filename (`<uuid4>.pdf`) preventing collisions.
size_bytes	INTEGER	None	NOT NULL	None	Exact file payload size in bytes for storage calculations.
created_at	TIMESTAMP WITH TIMEZONE	None	NOT NULL	UTC datetime.now	Upload timestamp in UTC.
user_id	VARCHAR(36)	FOREIGN KEY, INDEX	NULLABLE	None	References `users.id`; tenant ownership filter.
folder_id	VARCHAR(36)	FOREIGN KEY, INDEX	NULLABLE	None	References `folders.id`; null indicates uncategorized.

3.4 Table Name: `image_categories`

Defines user-specific taxonomy groups for digital image assets.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
id	VARCHAR(36)	PRIMARY KEY	NOT NULL	UUID4 hex string	Unique identifier for the image category.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
name	VARCHAR(120)	None	NOT NULL	None	Category title (e.g. 'Logos', 'Receipts', 'Screenshots').
user_id	VARCHAR(36)	FOREIGN KEY, INDEX	NOT NULL	None	References `users.id`; defines tenant ownership.
created_at	TIMESTAMP WITH TIMEZONE	None	NOT NULL	UTC datetime.now	Creation timestamp in UTC.

3.5 Table Name: `image_assets`

Maintains metadata for stored raster and vector image assets.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
id	VARCHAR(36)	PRIMARY KEY	NOT NULL	UUID4 hex string	Unique identifier for the image asset.
filename	VARCHAR(255)	INDEX	NOT NULL	None	Original user-facing image filename.
stored_filename	VARCHAR(255)	UNIQUE	NOT NULL	None	Physical disk filename (`<uuid4>.<ext>`).
content_type	VARCHAR(100)	None	NOT NULL	None	MIME type (e.g. `image/png`, `image/jpeg`).
size_bytes	INTEGER	None	NOT NULL	None	Image payload size in bytes.
category_id	VARCHAR(36)	FOREIGN KEY, INDEX	NOT NULL	None	References `image_categories.id`.

Column Name	Data Type	Constraints	Nullability	Default Value	Description & Usage
user_id	VARCHAR(36)	FOREIGN KEY, INDEX	NOT NULL	None	References `users.id`; tenant ownership filter.
created_at	TIMESTAMP WITH TIMEZONE	None	NOT NULL	UTC datetime.now	Upload timestamp in UTC.

4. Physical Storage & Filesystem Architecture

Binary files are segregated from the database and organized within the server filesystem under dedicated paths configured via `settings.storage_dir`:

- **PDF Storage Root:** `backend/storage/pdfs/` — Houses all uploaded PDF documents named as `<uuid4>.pdf`.
- **Image Storage Root:** `backend/storage/pdfs/images/` — Houses categorized image files named as `<uuid4>.<ext>`.
- **Path Traversal Mitigation:** Because stored files are renamed to pure hex UUIDs upon arrival, malicious filenames (e.g. `../../etc/passwd` or `upload.php.pdf`) have zero effect on filesystem directory structures.
- **Atomic Rollback Mechanics:** File uploads execute in a two-stage transactional pattern: the file stream is written to a temporary UUID target; if the subsequent database session commit fails or encounters an exception, an exception handler immediately triggers `target.unlink(missing_ok=True)` to prevent orphaned disk waste.

5. Indexing Strategy & Performance Optimization

To ensure sub-100ms response times at scale, database indexes are strategically placed on foreign keys and search columns:

Target Table	Indexed Field(s)	Index Type	Target Query Pattern	Performance Impact
users	username	B-Tree (Unique)	Authentication lookups by username	O(1) lookup during user registration checks.

Target Table	Indexed Field(s)	Index Type	Target Query Pattern	Performance Impact
users	email	B-Tree (Unique)	OAuth2 login lookups by email	O(1) lookup during login credential verification.
documents	user_id	B-Tree	`SELECT * FROM documents WHERE user_id = :uid`	Essential for fast multi-tenant filtering.
documents	filename	B-Tree	`WHERE filename ILIKE '%term%'`	Accelerates prefix searches and sorts.
documents	folder_id	B-Tree	`WHERE folder_id = :fid AND user_id = :uid`	Eliminates full table scans when filtering by folder.
image_assets	category_id	B-Tree	`WHERE category_id = :cid`	Instant listing of images within an active category.

6. Production Schema DDL (PostgreSQL & SQLite Compatible)

The complete Data Definition Language (DDL) for creating the production relational schema:

```
-- PDF Vault Enterprise Relational Schema (PostgreSQL 16+ Compatible)

CREATE TABLE IF NOT EXISTS users (
    id VARCHAR(36) PRIMARY KEY,
    username VARCHAR(80) NOT NULL UNIQUE,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);
CREATE INDEX IF NOT EXISTS ix_users_username ON users (username);
CREATE INDEX IF NOT EXISTS ix_users_email ON users (email);

CREATE TABLE IF NOT EXISTS folders (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    user_id VARCHAR(36) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);
CREATE INDEX IF NOT EXISTS ix_folders_user_id ON folders (user_id);
```

```
CREATE TABLE IF NOT EXISTS documents (
    id VARCHAR(36) PRIMARY KEY,
    filename VARCHAR(255) NOT NULL,
    stored_filename VARCHAR(255) NOT NULL UNIQUE,
    size_bytes INTEGER NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    user_id VARCHAR(36) REFERENCES users(id) ON DELETE CASCADE,
    folder_id VARCHAR(36) REFERENCES folders(id) ON DELETE SET NULL
);
CREATE INDEX IF NOT EXISTS ix_documents_filename ON documents (filename);
CREATE INDEX IF NOT EXISTS ix_documents_user_id ON documents (user_id);
CREATE INDEX IF NOT EXISTS ix_documents_folder_id ON documents (folder_id);

CREATE TABLE IF NOT EXISTS image_categories (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    user_id VARCHAR(36) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);
CREATE INDEX IF NOT EXISTS ix_image_categories_user_id ON image_categories (user_id);

CREATE TABLE IF NOT EXISTS image_assets (
    id VARCHAR(36) PRIMARY KEY,
    filename VARCHAR(255) NOT NULL,
    stored_filename VARCHAR(255) NOT NULL UNIQUE,
    content_type VARCHAR(100) NOT NULL,
    size_bytes INTEGER NOT NULL,
    category_id VARCHAR(36) NOT NULL REFERENCES image_categories(id) ON DELETE CASCADE,
    user_id VARCHAR(36) NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);
CREATE INDEX IF NOT EXISTS ix_image_assets_filename ON image_assets (filename);
CREATE INDEX IF NOT EXISTS ix_image_assets_category_id ON image_assets (category_id);
CREATE INDEX IF NOT EXISTS ix_image_assets_user_id ON image_assets (user_id);
```

7. Backup, Disaster Recovery & High Availability

To protect enterprise customer assets against corruption or data loss:

- **SQLite WAL & Hot Backups:** In development and single-node setups, SQLite is operated in WAL (Write-Ahead Logging) mode. Daily hot snapshots are generated via the SQLite Online Backup API.
- **Managed PostgreSQL Backups:** In production cloud environments (AWS RDS / Cloud SQL), automated daily snapshots with continuous WAL archiving enable Point-in-Time Recovery (PITR) up to 35 days.
- **Object Storage Replication:** When moving to S3/GCS blob storage, Cross-Region Replication (CRR) and Object Versioning provide protection against accidental deletion or regional cloud outages.